

Agent Reliability in Financial Tool Use: A Layered Orchestration Approach

Abstract

AI agents completing financial operations using tool-calling interfaces fail at a 75% rate out of the box. The failures are not in reasoning (the agent picks the correct tool) but in execution: malformed arguments and broken data chaining between steps.

This paper presents a layered orchestration architecture that brings the failure rate to zero through four complementary mechanisms: argument validation, structured retry, schema-rich tool descriptions, and typed output labeling. Each layer addresses a distinct failure category. No single layer is sufficient alone.

1. The Evaluation Gap

Existing LLM evaluation frameworks (Braintrust, Langsmith, Promptfoo, DeepEval) test prompt-level safety: can the model reject injection attacks, produce accurate outputs, follow instructions? These are necessary but miss the dominant failure mode in financial tool use: the agent calls the right tool with the wrong arguments.

Agent protocol frameworks (ACK, MPP, ERC-8004) define the tools and message formats but provide no evaluation infrastructure. They ship demos and unit tests.

None of them test whether an AI agent can autonomously complete a multi-step workflow using their tools, measure the failure rate, or provide an orchestration layer to improve it. To my knowledge, this is the first framework that evaluates agent reliability at the workflow level using real financial cryptographic operations, identifies the specific failure taxonomy, and provides a layered solution that achieves 100% completion.

Financial workflows are multi-step and cryptographically chained. A credential signing step produces a JWT that must be passed intact to a verification step.

A payment request must include exact field specifications (amount, decimals, currency, recipient) or the tool rejects it. These are not reasoning failures. They are execution failures, and they require a different evaluation and mitigation approach.

2. Failure Taxonomy

Testing Claude Sonnet 4 against four financial workflows (identity credential lifecycle, payment request lifecycle, untrusted issuer rejection, wrong issuer rejection) revealed three failure categories:

Category 1: Argument Construction. The agent omits required fields or passes wrong types. Example: creating a payment request without the `decimals` field, which is required by the payment schema.

The agent understands the workflow but cannot reliably populate all required fields from memory.

Category 2: Argument Chaining. The agent constructs valid arguments for each tool individually but fails to correctly pass output from one step as input to the next. Example: a signing step produces a 600-character JWT string, but the verification step receives a truncated or restructured version.

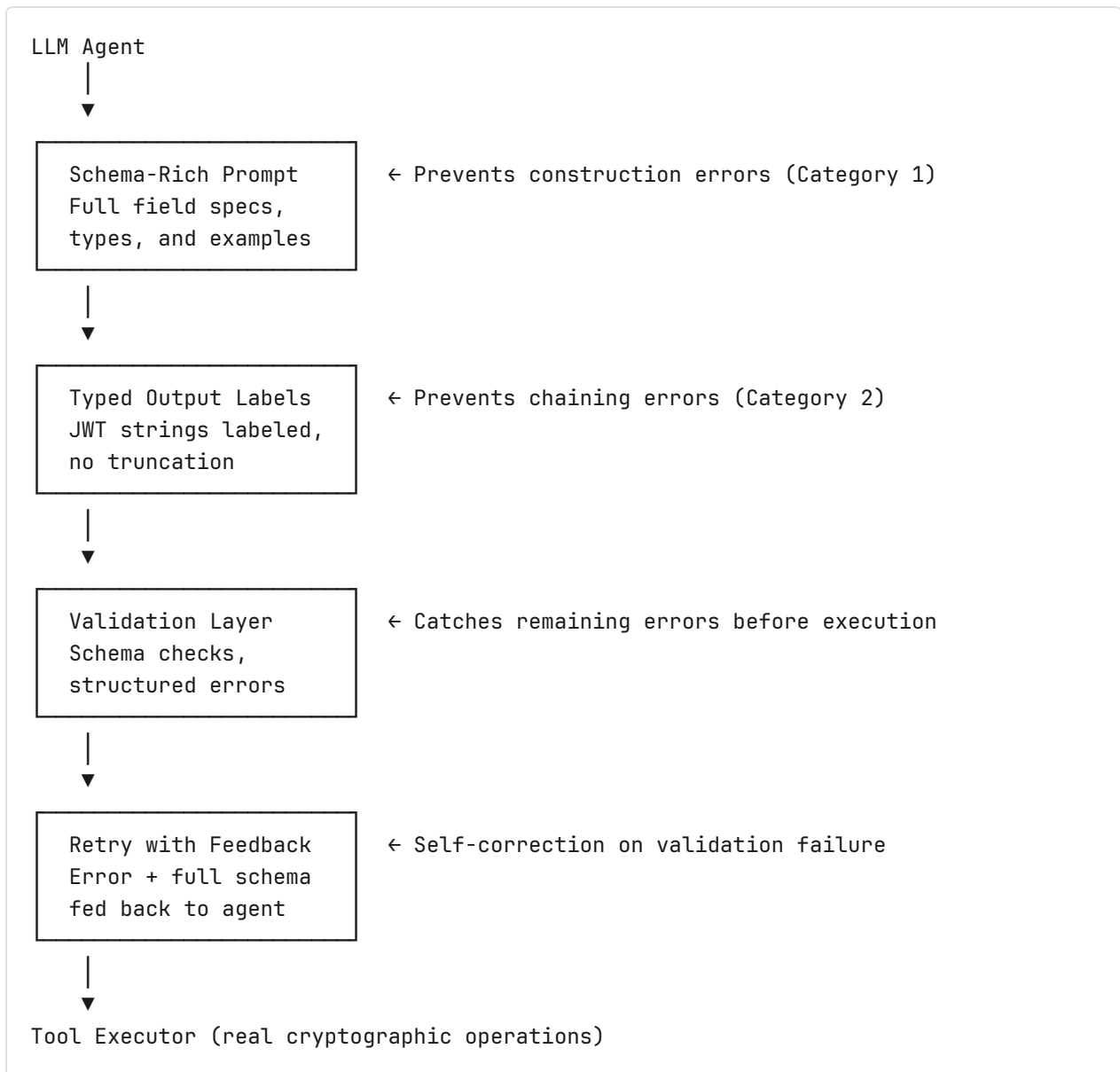
Cryptographic values are binary: one wrong character and verification fails.

Category 3: Workflow Reasoning. The agent selects the wrong tool or fails to recognize error states. This was never observed.

The model consistently selected the correct tool and correctly identified verification failures. Categories 1 and 2 account for all observed failures. Most evaluation frameworks only test Category 3.

3. The Layered Architecture

Four mechanisms, each addressing a different failure category:



3.1 Schema-Rich Prompting

Replace terse tool descriptions with complete field specifications including types, required markers, and examples. Instead of:

```
ack_create_payment_request: Args: { paymentOptions: [...], jwk, did }
```

Provide:

```
ack_create_payment_request
  Required: {
    "paymentOptions": [{
      "id": "option-1",
      "amount": 1.00,
      "decimals": 6,
      "currency": "USDC",
      "recipient": "0x1234...5678"
    }],
    "jwk": "<JWK string from keypair>",
    "did": "<requester DID>"
  }
  Note: every payment option MUST include id, amount, decimals,
  currency, recipient.
```

Impact: Eliminated all argument construction errors. Zero validation failures after this change.

3.2 Typed Output Labels

When presenting previous step results to the agent, label outputs by their data type rather than dumping raw text:

- JWT strings: "JWT string (pass this entire value as-is): eyJ..."
- JSON objects: passed through as-is
- Errors: "ERROR: ..."

Critically, do not truncate cryptographic values. JWTs are 400-800 characters.

Truncation produces cryptographically invalid input that will fail downstream verification. The agent correctly passes full JWT strings between signing and verification steps after this change.

3.3 Validation Layer

Check tool arguments against schemas before execution. When arguments are invalid, return a structured error message that includes the specific missing fields and the full required schema:

```
Validation failed. Fix ALL of these issues and retry:
- paymentOptions[0].decimals: Each payment option must have
  decimals (e.g. 6 for USDC)
```

```
Full required schema for ack_create_payment_request:
```

```
{
  "paymentOptions": [{ "id": string, "amount": any,
    "decimals": number, "currency": string,
    "recipient": string }],
  "jwk": string,
  "did": string
}
```

The validation layer also produces an audit log with timestamps, tool names, and field-level errors for every rejected call. The audit log provides compliance data showing exactly when and how the agent attempted malformed operations.

3.4 Retry with Feedback

When validation rejects a tool call, feed the error (including the failed attempt) back to the agent as context and allow additional attempts. The agent sees what it did wrong and the full schema of what's expected.

Impact: Allows self-correction. In testing, retry improved pass rate from 25% to 50% before the other layers eliminated the need for it.

4. Results

Configuration	Pass Rate	Validation Errors
Baseline (no layers)	25%	N/A
+ Validation	25%	2
+ Validation + Retry	50%	4 (across retries)
+ Improved Prompt	50%	0
+ Output Labeling	100%	0

All four layers together achieve 100% pass rate with zero validation errors on a suite of four financial workflows. These cover identity credentials, payment requests, failure recognition, and issuer verification.

5. Production Application

Deployment Architecture

The validation layer serves as both the evaluation tool and the production orchestration layer. The same code that evaluates agent reliability during development enforces it in production:

- Pre-execution schema validation catches malformed tool calls
- Structured retry allows self-correction without human intervention
- The audit log captures every validation failure for compliance
- The deterministic baseline (runs in under 1 second with no API calls) serves as a regression gate in CI

Evaluation Pipeline

1. **Pre-deployment:** Run deterministic baseline (must pass, proves tools work). Run LLM eval (sets reliability expectation).
2. **Post-update:** Re-run on model or tool changes. Regression is a blocking issue.
3. **Continuous:** Schedule evals against production endpoints. Track validation error rates for drift detection.
4. **Model comparison:** The framework is model-agnostic. Compare Claude, GPT-4o, and Gemini against the same workflows.

6. Closing Thoughts

As far as I can tell, no existing framework evaluates agent reliability at the workflow level using real financial cryptographic operations. The failure taxonomy here (argument construction, argument chaining, and workflow reasoning) came from observing actual failures, not from theory. Most eval frameworks only test the one category (reasoning) that never failed.

The eval tools and the production orchestration layer are the same code. The validation, retry, and output labeling that fix eval failures prevent production failures. The progression from 25% to 100% is reproducible and each layer's contribution is independently measurable.

There's plenty left to figure out. Multi-agent workflows, where two parties transact with their own credentials and verify each other, are the obvious next step. Adversarial tool use is an open question: can prompt injection via workflow context trick an agent into redirecting a payment or skipping verification?

The current suite tests 3-4 step workflows. Real financial operations involve 8-12 steps with branching logic. Different models may fail differently, and the taxonomy should be validated across providers. Latency budgets and cost-per-completion metrics are needed before any of this is production-ready.